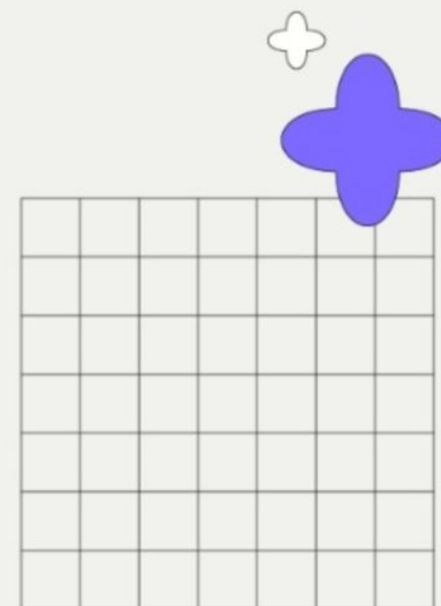
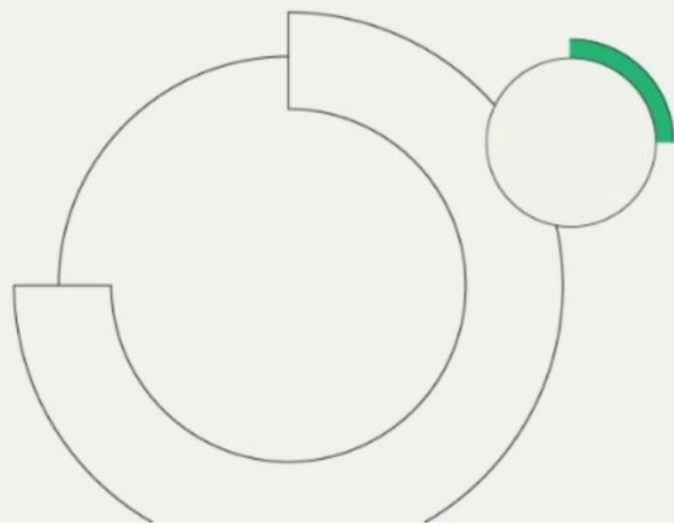
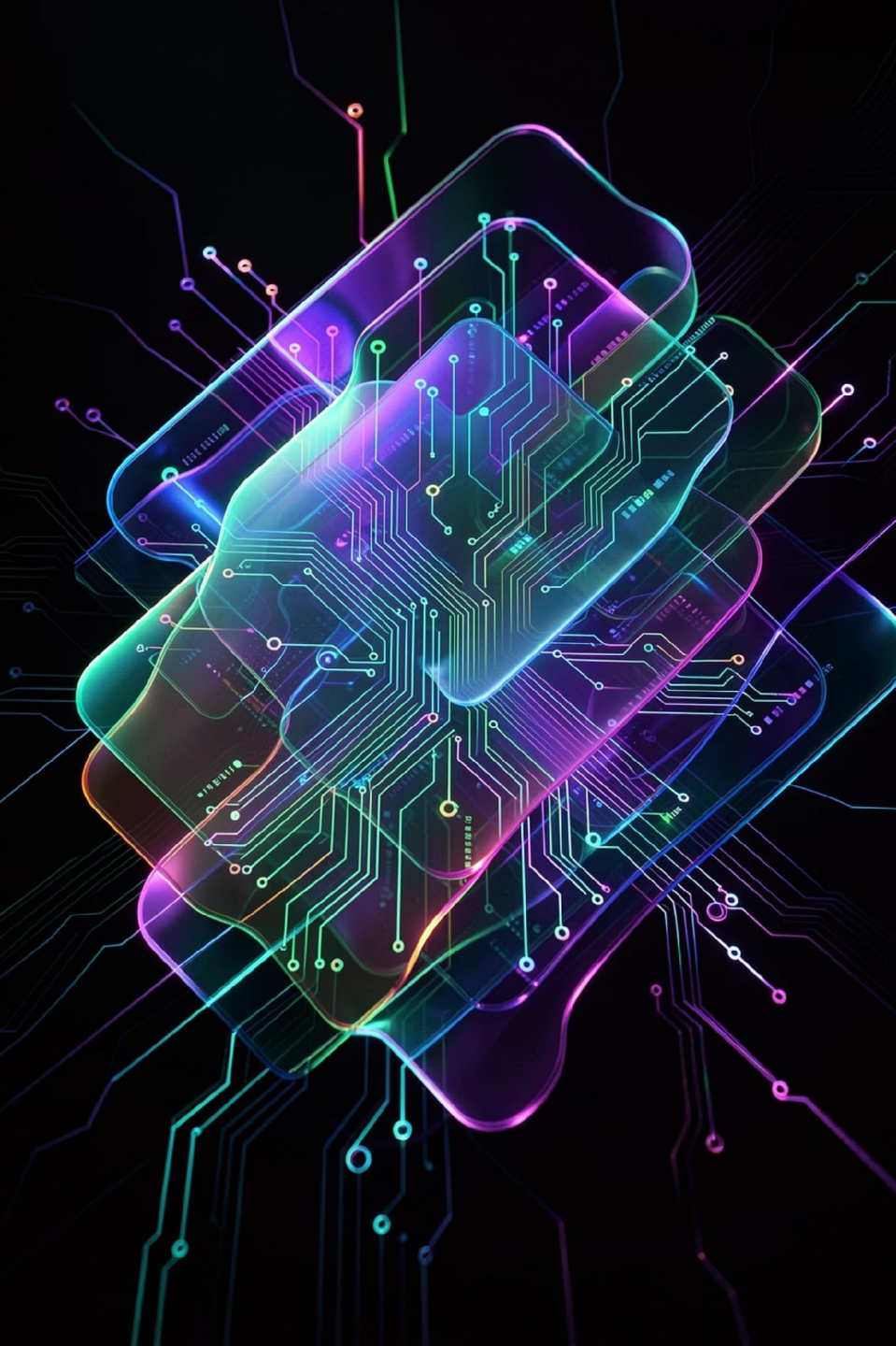




שיעור 9 - Prompt Engineering סיפורי משתמש - המשך



מסגור התשובה תבנית ה-E2E

משימוש בטכניקות לתהליך סדור
ליצירת סיפורי משתמש

בשיעור הזה: השיטה, הסדר, התבנית, והלופ

סיכום השיעור הקודם

מה למדנו

למדנו ארבע טכניקות: Zero-Shot, Persona, Chain of Thought וניהול הקשר.
כל טכניקה ייצרה סיפור משתמש שלם מקצה לקצה.

למה זה קרה

המודל מאומן להשלים משימות עד הסוף. (לבנות הכל E2E כמונוליט. ברגע שיש לו את הקשר של פרויקט ואת הפרסונה, הוא רץ קדימה.

המסקנה

העוצמה של המודל מחייבת אותנו לתחום אותו.
זה ההבדל המהותי בין מי שמשתמש ב AI למי שמנהל אותו.



השיטה: מסגור התשובה

שורת סיום קבועה

"סיים כל תשובה בשורה: 'זהו תוצר השלב. השלב הבא ממתין לבקשה שלך.'"

תוצר מוגדר בלבד

"הפק אך ורק את התוצר שהוגדר במשימה, ועצור בסופו".

ציון אפשרות התקדמות

"זיהית שאפשר להתקדם?
ציין זאת במשפט אחד, וההחלטה נשארת אצלנו".

בקשה מפורשת בלבד

"כל השלבים הנוספים נוצרים רק לאחר בקשה מפורשת בלבד".

הגדרה:

קובעים מראש, בתוך הפרומפט עצמו, את גבולות התוצר. מה מפיקים, היכן עוצרים, ומה ממתין לבקשה הבאה.

המסגור נשען על שני מרכיבים מאנטומיית הפרומפט:
אילוצים ופורמט.

בלוק המסגור נכנס להנחיות הפרויקט

תזכורת - מבנה של פרומפט אפקטיבי

פרומפט טוב בנוי ממרכיבים ברורים. לא חייבים את כולם בכל פרומפט.

משימה



מה בדיוק אנחנו מבקשים
שייעשה.

הקשר



כל הרקע שהמודל צריך כדי
להבין את המשימה.

תפקיד



מי המודל אמור להיות.
"פעל כמיישם AI, או אתה
פסיכולוג ארגוני".

אילוצים



מה אסור, מה חובה, מהם הגבולות. בעידן AI
המודלים רצים קדימה, האילוץ קובע היכן עוצרים

פורמט פלט



איך צריכה להיראות התשובה: מסמך MD, טבלה,
רשימה, קוד.

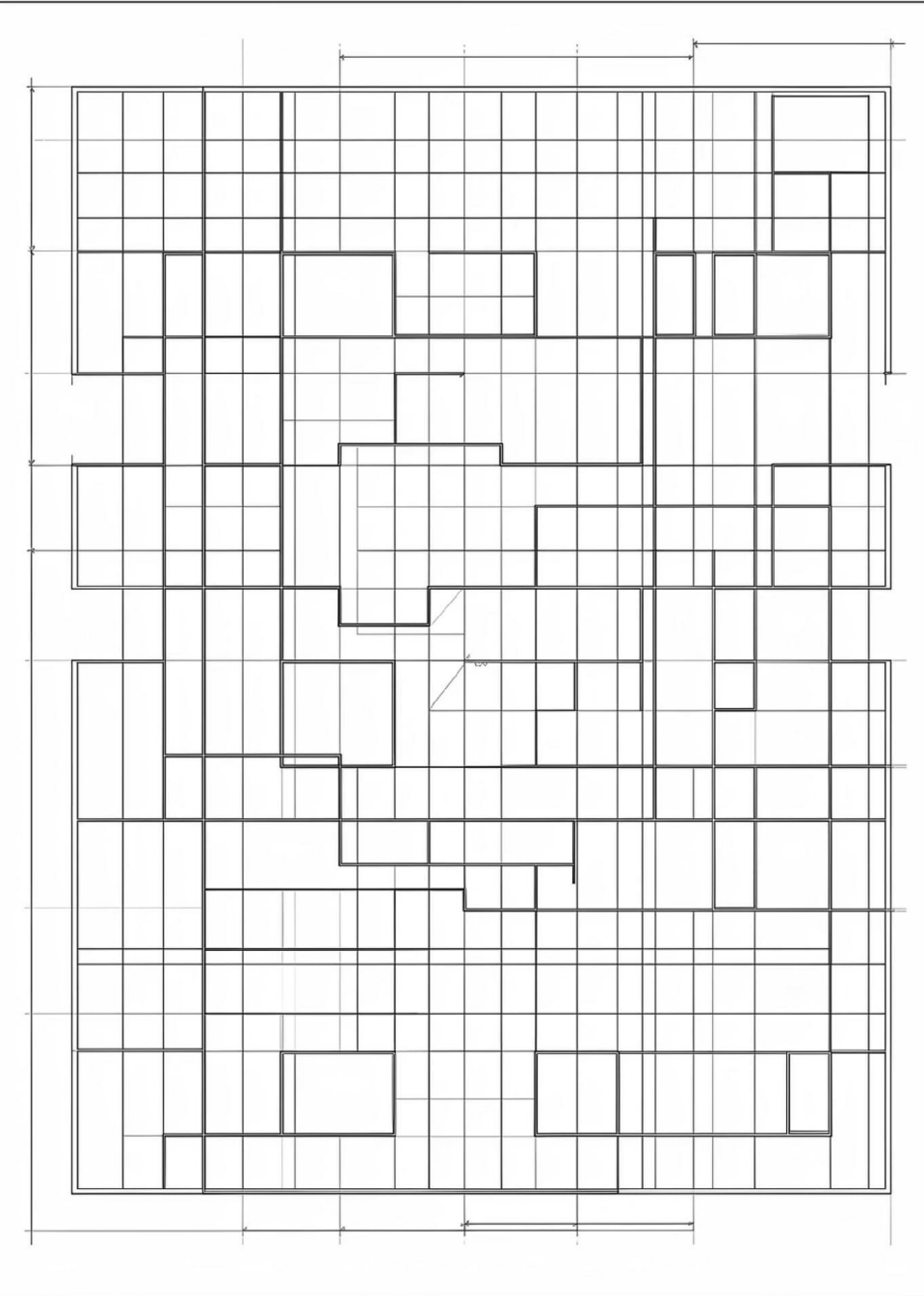
העקרון של השימוש בפרומפטים

התהליך
קובע **מה** בונים

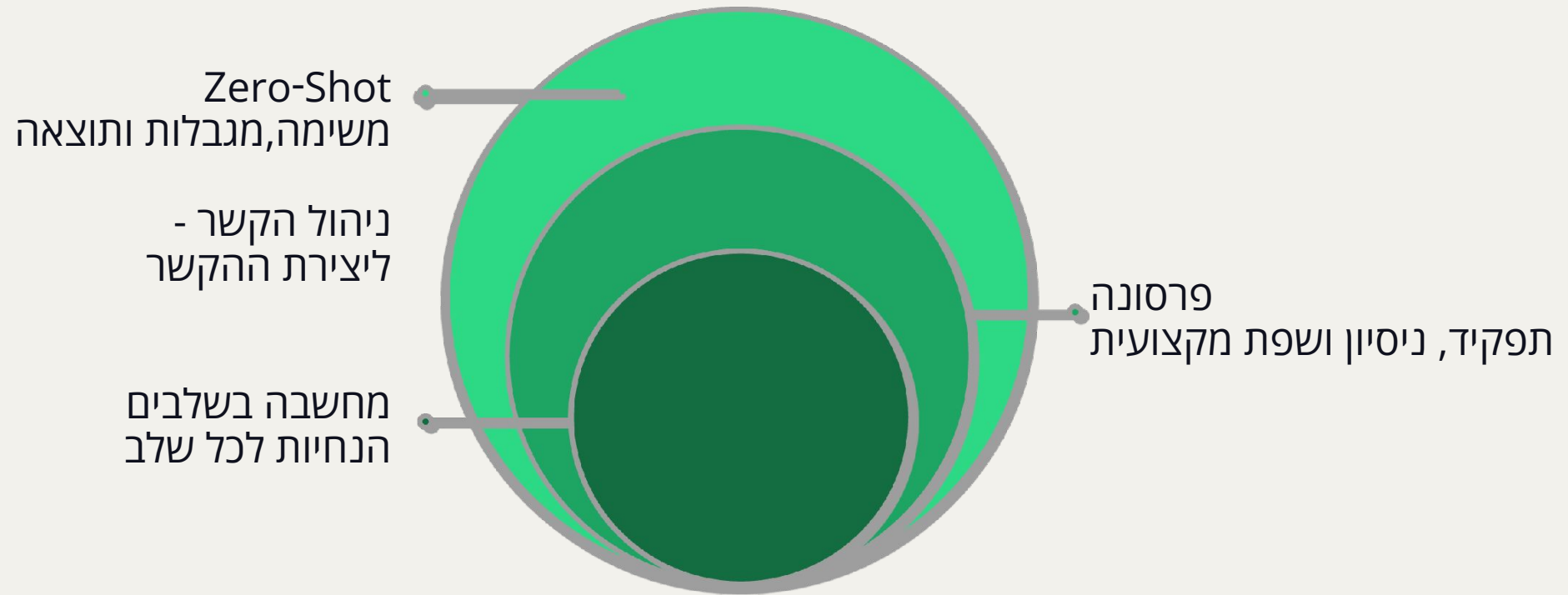
הטכניקה- הפרומפט
קובעת **איך** מנחים את המודל לבנות את זה

שלב בתהליך שלנו = תוצר אחד בלבד.
טכניקה = דרך אחת מסויימת להגיע אליו.

המבחן: בסוף אתם יודעים לומר באיזה שלב של התהליך
אתם נמצאים, ומה הטכניקה עשתה בתוצר של השלב.



ארבע הטכניקות שהשתמשנו בהן



אנחנו בונים סיפור משתמש ראשון.
הדרישה עליה אנחנו עובדים במערכת התקלות - עובדים פוחדים לדווח כי ההנהלה מחפשת אשמים בתקלות.

סיפור משתמש - User Story מורחב

6 מרכיבים להצלחה **המבנה החדש**

1

הפרסונה - AS -
מי אני - המשתמש (הפרסונה)

4

אני רוצה - I WANT -
מה אני צריך (הפתרון)

2

למה זה נדרש - SO THAT -
למה אני צריך (הערך או צמצום
בזבוז)

5

קריטריוני הצלחה -
- ACCEPTANCE CRITERIA
איך נדע שהצלחנו

3

ההקשר (CONTEXT) -
ההקשר והמצב הנוכחי

6

דוגמאות - EXAMPLES -
דוגמאות ותרשימים

איך זה עובד? כל מרכיב עונה על שאלה קריטית

ביחד - תמונה מלאה!



WANT I (הפתרון)

מה בדיוק צריך לעשות?

WANT I צריך להיות ספציפי למשתמש צורך או משתמש רצון

חייב להיות ברור ומדויק

זה מה שבונים בפועל!

צריך לתאר את כל השלבים

מה אנחנו רוצים לבנות?

מערכת ניהול **תקלות** מבוססת AI

1

דיווח:

מהנייד בלחיצה

AI מבצע את כל התהליך

2

הקטנת הפחד: **הדרישה בה**

אנחנו מטפלים

הסתרת זהות המדווח, קרדיט

למדווח, מונה קרדיטים אישי

3

דשבורד:

איחוד תקלות

לוח מצטיינים שבועי וחודשי

מגמות

4

בקרת איכות:

דיווח נסגר רק לאחר אימות

סגירת מעגל

הקטנת חזרתיות תקלה



איך כותבים I WANT טוב?

מבנה אפשרי:

2

כולל:

- [רכיב 1]
- [רכיב 2]
- [רכיב 3]

1

I WANT

[פעולה ראשית]

3

ש-[תנאי/מגבלה]



דוגמה - מערכת תקלות

❌ רע:

I WANT

שהמערכת תאפשר
להעלות תקלות ולהעביר
אותן לטיפול האחראי

✅ טוב:

I WANT מערכת שתאפשר לעובד לדווח על תקלה, כולל תקלה שהוא עצמו גרם,
בלי שזהותו תיחשף למנהליו.

כולל:

כל עובד שחושש מחיפוש אשמים יוכל לדווח בביטחון באופן אנונימי.

- דיווח שמסומן אנונימי
- הסתרת הזהות מכל בעל תפקיד ניהולי.
- גישה לשם המדווח רק לעובד עצמו ולבעלי הרשאה מוגדרים.
- רישום כל ניסיון גישה לזהות בלוג ביקורת.

הבדיקה: "האם מפתח יכול לבנות את מה שנדרש מהתיאור שלנו?"



טכניקת Recipe Pattern

שיטה יעילה ליצירת תבניות קבועות לפעולות חוזרות.

ה Recipe Pattern מבטיח עקביות, שלמות ועילות בתהליכי

עבודה סטנדרטיים ומתאים במיוחד לאפיון סיפורי משתמש,

מערכות, כתיבת מסמכי תיעוד, וניתוחים מובנים.

- נראה כיצד Recipe Pattern יוצר את ה-WANT I מתוך הערך

שהוגדר ב-SO THAT והמשתמשים

כיצד יוצרים: Recipe

1

הגדירו את המטרה
הערך או הבוזז שכבר נוסחו ב-SO
THAT [הערך או הבוזז]
+הוסף משתמשים רלוונטיים].

2

הוסיפו רכיבים ידועים
[כל מה שברור מהדרישות
ומהאבחון].

3

הגדירו מה המודל משלים
[השלם רכיבים חסרים בנוסף
לידועים].

4

הגדרת רכיבי הפתרון
[הגדר את הרכיבים. כל רכיב חייב
לשרת את המטרה.
כתוב אותם כ I WANT | מפורט
שמפתח יוכל לבנות מהם מערכת
שהוגדרה].

המבחן: אם המודל הציע רכיב שאינו משרת את המטרה, תנאי
האיכות בפרומפט חלש מדי.



עכשיו תורכם

הגדירו את ה- **WANT** של הפרויקט שלכם באמצעות טכניקת Recipe Pattern



סיפור משתמש - User Story מורחב

6 מרכיבים להצלחה **המבנה החדש**

1

הפרסונה - AS -
מי אני - המשתמש (הפרסונה)

2

למה זה נדרש - SO THAT -
למה אני צריך (הערך או
צמצום בזבז)

3

ההקשר (CONTEXT) -
ההקשר והמצב הנוכחי

4

אני רוצה - I WANT -
מה אני צריך (הפתרון)

5

קריטריוני הצלחה -
ACCEPTANCE CRITERIA
איך נדע שהצלחנו

6

דוגמאות - EXAMPLES -
דוגמאות ותרשימים

איך זה עובד? כל מרכיב עונה על שאלה קריטית

ביחד - תמונה מלאה!



קריטריוני הצלחה (ACCEPTANCE CRITERIA)

Fact Check List טכניקות

טכניקת Fact Check List

טכניקות הגנתיות

מבקשים מהמודל לסמן, לכל נתון וטענה בתוצר:
מאומת, מסתבר, או לא ידוע.



קריטריוני ההצלחה

זה המקום שבו ה-AI ממציא מספרים בביטחון מלא:
"זמינות 99 אחוז, דיוק 95 אחוז". כל מספר שנכנס לסיפור
בלי אימות הופך ליעד שהמערכת תיבחן מולו.

כלל הזהב

כל מספר חייב להיות מסומן לפני שהוא נכנס לסיפור.

Fact Check List: כיצד יוצרים:



הגדירו מקורות אימות
נתוני האבחון, מחקר השוק, מדידות
מהשטח.



הגדירו שלוש קטגוריות
מאומת, מסתבר, לא ידוע.



קבעו את הכלל
נתון בלי סיווג אינו נכנס לתוצר.



בקשו טבלה מסודרת
שלוש עמודות: הנתון, הסיווג, ומה
נדרש כדי לאמת.

המבחן: כל "לא ידוע" בטבלה מחייב בדיקה. את מסתבר- חייבים לבחון לעומק

[illegible]

התבנית Fact Check List לקריטיוני הצלחה

על בסיס הסיפור שכתבנו (הקשר, משתמש, ערך, פתרון):

1

נסח קריטריוני קבלה מדידים
בקטגוריות: ביצועים, דיוק,
אנונימיות, תוצרים.

2

סמן כל מספר שאתה מציע
מאומת - מתוך נתוני הפרויקט.
מסתבר - הערכה סבירה שדורשת
וידוא.
לא ידוע - חייב בדיקה.

3

ציין מה נדרש אימות
לכל "מסתבר" ו"לא ידוע", ציין מה
נדרש כדי לאמת.

4

הצג הכל כטבלה: [כמות עמודות ושמות העמודות]
צור מבנה אחיד, ברור ומסודר לכל הקריטריונים וסיווגיהם.

דוגמה - מערכת התקלות: הקריטריונים אחרי הסיווג

קריטריון	סיווג	אימות נדרש
הסתרת זהות מכל בעל תפקיד ניהולי, 100%	מאומת 	דרישת ליבה
רישום כל ניסיון גישה לזהות ביומן ביקורת, 100%	מאומת 	דרישת ליבה
אחוז העובדים שמדווחים על תקלה שהם גרמו, יעד מ-5% ל-40% תוך שנה	מסתבר 	עד למפעל, אימות במעקב
עלייה בכמות דיווחי Near-miss, מ-3 ל-30 ל-100 עובדים	מאומת  מסתבר 	מאומת כבנצ'מרק ממחקר השוק, מסתבר כיעד. האימות: מעקב רבעוני



ACCEPTANCE CRITERIA (קריטריוני קבלה)

איך נדע שהצלחנו? OKRs ו- KPIs

כללי זהב לכתובת קריטריונים טובים?

מדיד - אפשר למדוד/לבדוק

ספציפי - לא מעורפל

ניתן לבדיקה - אפשר לוודא שהצליח

ריאלי - בר השגה בזמן הנדרש

למה CRITERIA חשובים?

- מגדירים "עבר הצלחה" בצורה ברורה

- מונעים אי הבנות

- מדידים וניתנים לבדיקה

דוגמאות למדדים טובים- דיווח אנונימי

1

טוב: ✓

ACCEPTANCE CRITERIA

- הסתרת זהות המדווח מהמנהלים
- ב-100% מהמקרים, "עלייה מ-5% ל-40% בעובדים שמדווחים על תקלה שהם גרמו תוך שנה",
- "כל ניסיון גישה לזהות נרשם ביומן"

למה זה טוב?

- יש מספרים קונקרטיים
- 100% מהמקרים אנונימיים
- עליה ל-40% דיווח
- שקיפות

2

רע: ✗

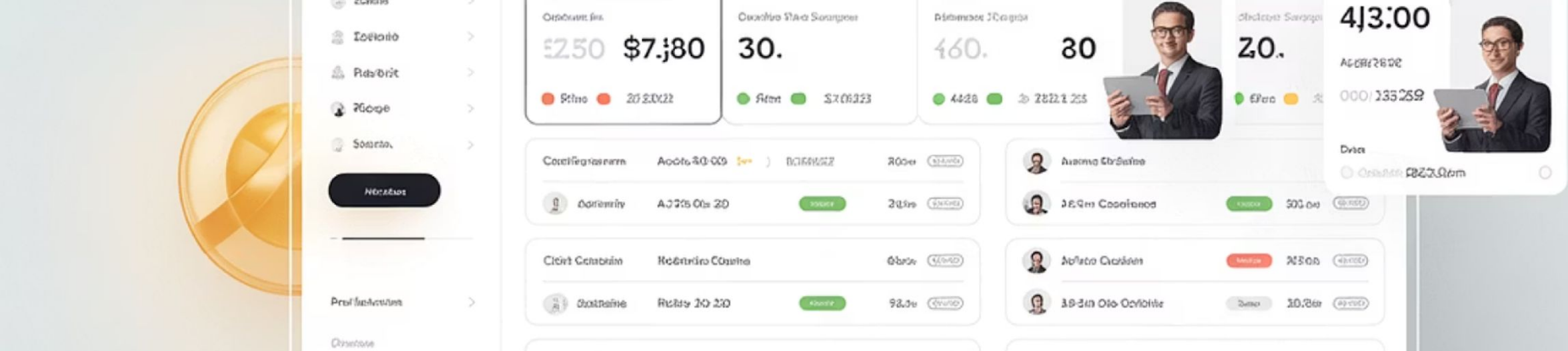
SO THAT

"תהליך הטיפול יהיה אנונימי"

למה זה רע?

איך בודקים אנונימיות

- לא מדיד ✗
- אין ערך ברור ✗



תוצרים נדרשים

- דיווח המסומן אנונימי
- זהות מוסתרת מכל בעל תפקיד ניהולי
- גישה לזהות רק לעובד ולבעלי הרשאה
- יומן ביקורת לכל ניסיון גישה
- אישור למדווח שהדיווח נקלט ללא שם

עכשיו תורכם

הגדירו את קריטריוני הקבלה בפרויקט שלכם.



נסחו את קריטריוני ההצלחה של הפרויקט שלכם לדרישה ספציפית
בעזרת Fact Check List.

מסתבר 🟡

הערכה סבירה - דורשת
וידוא לפני שנכנסת
לסיפור

מאומת ✅

נתון שמגיע ישירות מנתוני
הפרויקט

? לא ידוע

דורש בדיקה - מקבל גם משימת אימות מפורשת

כל מספר בטבלה מקבל סיווג. קריטריון שכולו "לא ידוע" מקבל
גם משימת אימות.



סיפור משתמש - User Story מורחב

6 מרכיבים להצלחה **המבנה החדש**

3

ההקשר (CONTEXT) -
ההקשר והמצב הנוכחי

2

למה זה נדרש - SO THAT
למה אני צריך (הערך או צמצום
בזבוז)

1

הפרסונה - AS
מי אני - המשתמש (הפרסונה)

6

דוגמאות - EXAMPLES -
דוגמאות ותרשימים

5

קריטריוני הצלחה -
- ACCEPTANCE CRITERIA
איך נדע שהצלחנו

4

אני רוצה - I WANT
מה אני צריך (הפתרון)

איך זה עובד? כל מרכיב עונה על שאלה קריטית

ביחד - תמונה מלאה!

טכניקת Outline Expansion

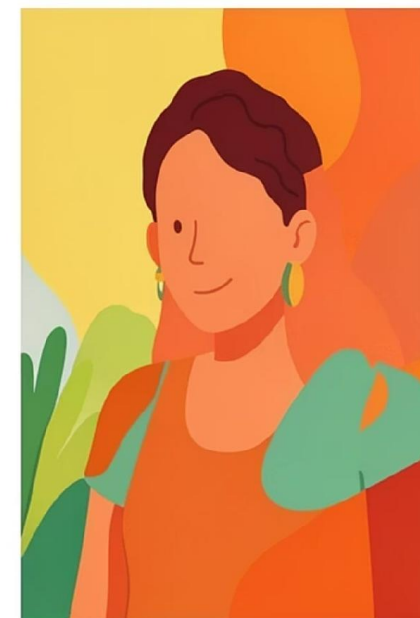
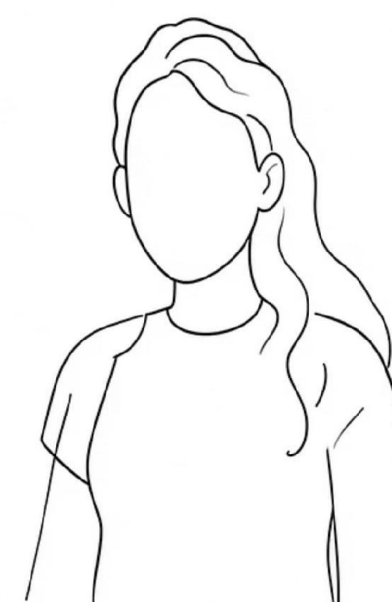
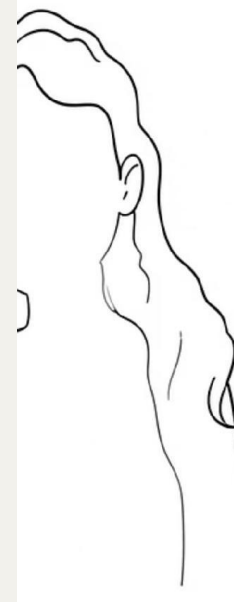
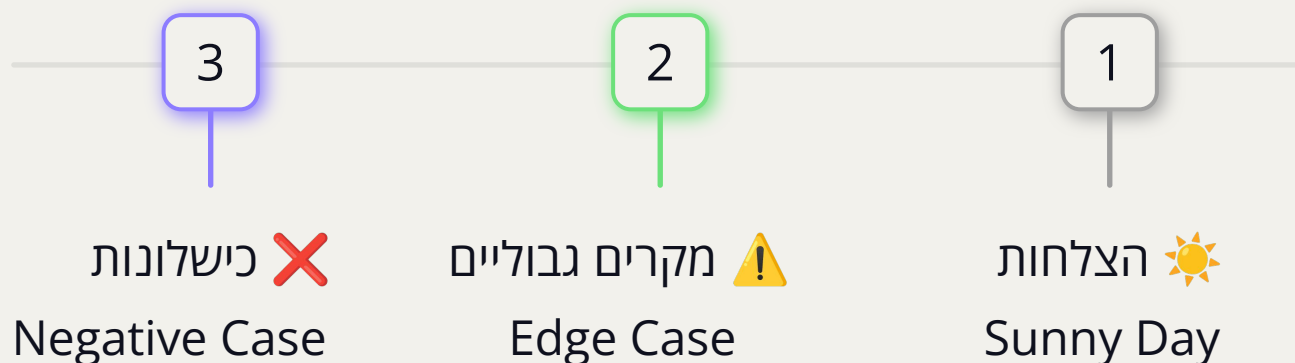
מתחילים משלד, ומרחיבים סעיף אחר סעיף עד לפירוט מלא.

למה תרחישים?

לתרחישים יש שלד קבוע: הצלחות, מקרים גבוליים, כישלונות. ניסיון לכתוב את שלושתם יחד מייצר תרחישים שטחיים. הרחבה של כל אחד מהם מייצרת עומק.

העיקרון המנחה

השלד נשאר קבוע בכל סיפור, והפירוט משתנה לפי המערכת. כך מובטח כיסוי מלא של כל מקרי הקצה.



כיצד יוצרים: Outline Expansion

1

קבעו את השלד

הסעיפים הראשיים
הקבועים.
בתרחישים: הצלחות
מקרים גבוליים
כישלונות.

3

הרחבה בכל פעם

עצירה לאישור לפני ההרחבה הבאה.

2

מגדירים לכל סעיף

- מצב פתיחה
- פעולת המשתמש
- תגובת המערכת
- פלט צפוי



העצירה מהותית: היא שומרת עליכם בשליטה ומונעת עשרה עמודים שאיש לא ביקש.

תבנית Outline Expansion לתרחישים

להלן שלד התרחישים של הסיפור:

1. הצלחה (Sunny Day) - אחד לפחות

2. מקרה גבולי (Edge Case) - אחד לפחות

3. כישלון (Negative Case) - אחד לפחות

3

בדוק כל דרישה ספציפית
עבוד לפי פורמט גרמין

2

עצור לאישור
עצור אחרי כל תרחיש לאישור
לפני שתמשיך לבא אחריו. כך
נמנעים מתרחישים שטחיים.

1

הרחב כל תרחיש בנפרד
אחד אחרי השני, לפירוט מלא
הכולל: מצב פתיחה, פעולת
המשתמש, תגובת המערכת,
והפלט הצפוי.

תרחיש הכישלון הוא החשוב מאוד - הוא בודק שהמערכת מתפקדת כפי שנדרש.

פורמט Gherkin, שפת התרחישים

תרחיש טוב מורכב משלוש שורות שכל מפתח ובודק קוראים באותו אופן. -
Gherkin - שפת ה-BDD (פיתוח מונחה התנהגות):

Given

המצב לפני
הפעולה.
מה קיים
מי קשור
איזה נתון כבר
במערכת.

When

הפעולה עצמה.
מה המשתמש
עושה, רצוי
פעולה אחת
ברורה.

Then

תגובת המערכת.
מה המערכת
מחזירה, מציגה,
שומרת או
חוסמת.
כל התגובות

שלושת סוגי התרחישים נשמרים כשלד:

(Sunny Day), מקרה גבולי (Edge Case), כשל או חריג (Negative Case).
בכל סוג יכולים להיות כמה תרחישים



דוגמה- הדרישה

עובד מזהה תקלה או טעות, בארגון שלא סובלני לטעויות

הוא חושש לדווח בשמו.

המערכת נועדה לאפשר את הדיווח הזה למרות הפחד.

- התרחישים בודקים את הדרישה, במצבים שונים.
- בכל קטגוריה לפחות תרחיש אחד, רצוי יותר לכל מצב של אותה דרישה.



דוגמה: הדרישה לדיווח למרות הפחד - הצלחות

הצלחה, Sunny Day, תרחיש שני

Given עובד מזהה תקלה שחבר שלו גרם, וחושש להלשין

When הוא מדווח אנונימית על התקלה

Then המערכת קולטת את הדיווח בלי לקשר אותו למדווח או למי שגרם

And מציגה את התקלה למטפלים בלי שם

הצלחה, Sunny Day, תרחיש ראשון

Given עובד מזהה תקלה שהוא עצמו גרם לה, וחושש להסתבך

When הוא מדווח דרך המערכת ומסמן "דיווח אנונימי"

Then המערכת קולטת את הדיווח, מסתירה את זהותו מהמנהלים

And שולחת אישור שהדיווח התקבל ללא שם

דוגמה: הדרישה דיווח למרות הפחד - מקרים גבוליים

מקרה גבולי, Edge Case, תרחיש שני

Given עובד רוצה לדעת מה עלה בגורל הדיווח האנונימי שלו

When הוא נכנס למערכת לבדוק את הסטטוס

Then המערכת מציגה לו את מצב הדיווח שלו בלבד, מזוהה מולו

And שומרת על הזהות מוסתרת כלפי כל בעל תפקיד ניהולי

מקרה גבולי, Edge Case, תרחיש ראשון

Given עובד דיווח אנונימית, והמנהל זקוק לפרט טכני נוסף כדי לטפל

When המנהל מבקש הבהרה על הדיווח

Then המערכת מעבירה את השאלה למדווח בלי לחשוף מי הוא

And מאפשרת לו להשיב או להתעלם, לפי בחירתו

דוגמה: הדרישה דיווח למרות הפחד - כשלים

כשל, Negative Case, תרחיש שני
Given מנהל מנסה להצליב שעת דיווח עם יומן
הנוכחות כדי לזהות את המדווח
When הוא מבקש את שעת הדיווח המדויקת
Then המערכת מציגה טווח זמן מעורפל בלבד,
לא חותמת מדויקת
And רושמת את הבקשה ביומן הביקורת

כשל, Negative Case, תרחיש ראשון
Given מנהל משמרת מחובר למערכת ורוצה
לדעת מי דיווח על תקלה במשמרת שלו
When הוא מנסה לאתר את זהות המדווח
Then המערכת חוסמת את הגישה לזהות
And מציגה את מדיניות האנונימיות
And רושמת את הניסיון ביומן הביקורת

תבנית ריקה למילוי עצמי Gherkin

סיפור המרכזי של הפרויקט שלכם, נסחו את התרחישים לפי השלד.
מלאו תרחיש הצלחה אחד, וכמה תרחישים גבוליים וכמה תרחישי כשל
ככל שהמערכת דורשת:

מקרה גבולי, Edge Case

(תרחיש לכל מקרה קצה)

Given _____

When _____

Then _____

הצלחה, Sunny Day

Given _____

When _____

Then _____

כשל או חריג, Negative Case

(תרחיש לכל דרך כשל)

Given _____

When _____

Then _____

כמה תרחישים בכל קטגוריה? הרחיבו עד שאין מקרה קצה/כשל שלא כוסה.
תרחיש הכשל בודק כיצד המערכת מגנה על עצמה מפני התנהגות משתמשים.

עכשיו תורכם

הגדירו את **התרחישים והדוגמאות** בפרויקט שלכם. לפחות אחד מכל סוג



תרגיל עצמאי

הרחיבו את שלושת התרחישים של הסיפור המרכזי בפרויקט שלכם, בעזרת Outline Expansion.

1 תרחיש הצלחה (Sunny Day) - אחד לפחות מצב פתיחה, פעולת המשתמש, תגובת המערכת, הפלט הצפוי.

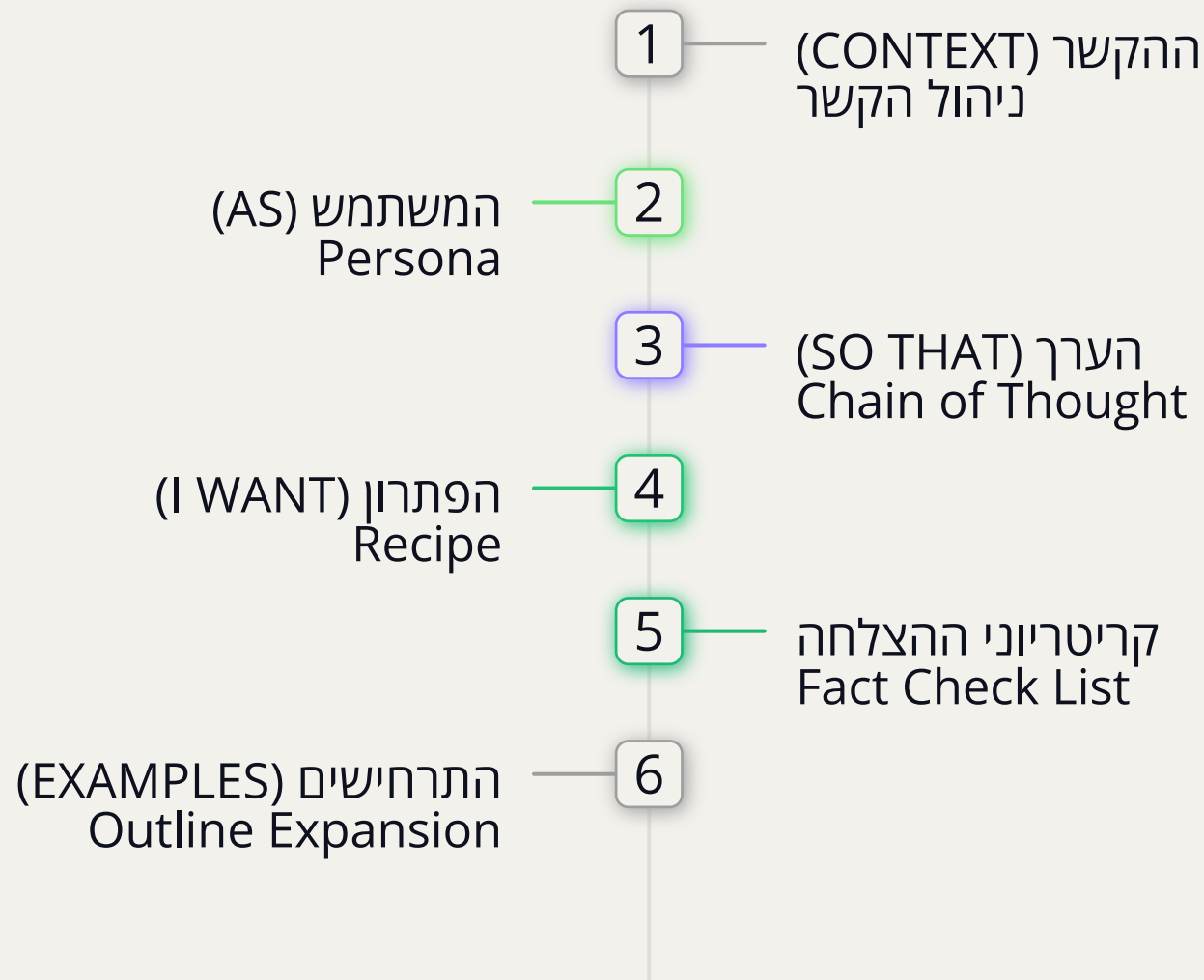
2 מקרה גבולי (Edge Case) - אחד לפחות מצב פתיחה, פעולת המשתמש, תגובת המערכת, הפלט הצפוי.

3 תרחיש כישלון (Negative Case) - אחד לפחות מצב פתיחה, פעולת המשתמש, תגובת המערכת, הפלט הצפוי.



השרשרת המלאה: שישה מרכיבים, שש טכניקות

כל מרכיב קיבל את הטכניקה שמתאימה לאופי החשיבה שלו. זו ערכת העבודה המלאה לכתיבת סיפור משתמש עם AI.



תבנית ה-E2E: הרכבה בדיעבד

כל השכבות שאנחנו יוצרים מתחברות לפורמט קבוע אחד: פרומפט שמקבל דרישה ומחזיר סיפור משתמש שלם.

הקשר
ממסמכי הפרויקט

פרסונה
תפקיד, ניסיון, מיקוד

מסגור
סיפור אחד ועצירה

MD פורמט + Few-Shot
משבצת דוגמה ופורמט פלט
קבוע

שרשרת חשיבה
בונה את ששת המרכיבים לפי
הסדר

שדה קלט משתנה
הדרישה והמשתמש - היחיד
שמשתנה בין ריצות

שדה הקלט הוא היחיד שמשתנה בין ריצות, ולכן התוצר יציב.



הכל ביחד יוצר סיפור משתמש אחד

.USER STORY



E2E

בניית פרומפט E2E ושיעורי הבית

נשלב את כל מה שלמדנו לפרומפט אחד יציב, עקבי ומוכן לשימוש

מה עשינו עד כה

עבדנו שלב אחרי שלב: לכל מרכיב השתמשנו בטכניקה שונה. כעת ששת המרכיבים שמורים בפרויקט ומוכנים להרכבה.

1

הקשר - CONTEXT
נוצר עם ניהול הקשר - שמרנו ברמת הפרויקט

2

AI תפקידו של - AS
נוצר עם Persona

3

הערך/ הבזבז - SO THAT
נוצר עם Chain of Thought

4

הפתרון - I WANT
נוצר עם Recipe Pattern

5

ACCEPTANCE CRITERIA
נוצר עם Fact Check List

6

תרחישים - EXAMPLES
נוצרו עם Outline Expansion

עכשיו: נהפוך את כל המרכיבים האלה לפרומפט אחד שעובד

תבנית הפרומפט E2E

1

[מסגור]

הפק סיפור משתמש אחד בלבד. עצור בסופו. סיים בשורה: "זהו תוצר השלב. השלב הבא ממתין לבקשה שלך."

2

CONTEXT

[ניהול הקשר - מהנחיות הפרויקט]
[ההקשר השמור בפרויקט]

3

AS [Persona]

עם ניסיון AI אתה מומחה לאפיון מערכות בכתובת סיפורי משתמש.

4

SO THAT

[Few-Shot] [דוגמה של סיפור משתמש מוכן מהשיעור]
[שדה קלט משתנה - היחיד שמשתנה בין ריצות] הדרישה: __ המשתמש: __

5

I WANT

התייחס: Recipe Pattern-השתמש ב כלול את כל, SO THAT-למטרה מ מפורט I WANT-הרכיבים הידועים, כתוב כ שמפתח יכול לבנות ממנו

6

ACCEPTANCE CRITERIA

סמן כל: Fact Check List-השתמש ב מספר מאומת / מסתבר / לא ידוע

7

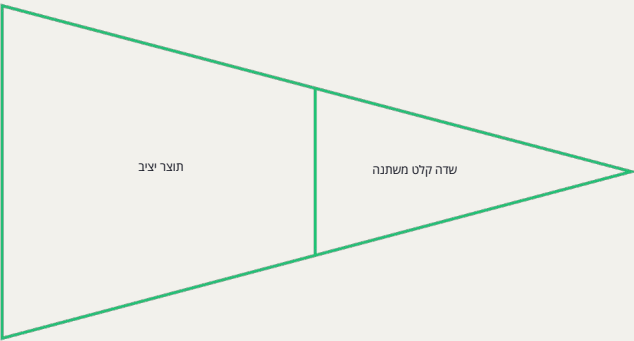
EXAMPLES

השתמש ב-Outline Expansion בפורמט Gherkin: הצלחה, מקרה גבולי, כישלון

למה הפרומפט הזה עובד

כל שכבה בפרומפט נועלת סוג אחר של כשל. יחד הן יוצרות מנגנון הגנה שמבטיח תוצר יציב בכל ריצה.

מה היא מונעת	שכבה
ריצה קדימה לסיפור הבא	מסגור
תוצר גנרי וחסר הקשר	פרסונה
ניחוש של המודל	הקשר
חשיבה שגויה או חסרה	שרשרת חשיבה
מספרים לא מאומתים	Fact Check List
תרחישים שטחיים	Outline Expansion



שדה הקלט הוא **היחיד שמשתנה** בין ריצות - לכן התוצר תמיד יציב ועקבי.

שיעורי בית

מה עליכם לעשות

הריצו את הפרומפט E2E על **כל הדרישות** ברשימה שלכם.
צרו סיפור משתמש לכל דרישה
עבדו לפי מסמך ההנחיות

תוצר - סיפור משתמש מלא לכל דרישה ברשימה - כולל קריטריוני הצלחה ותרחישים בפורמט Gherkin.

מה נעשה בשיעור 10

חזרה מהירה על הסיפורים ← כניסה לתרשימי זרימה ומקרי שימוש

עבודת בית

על בסיס המסמך, צרו לכל דרישה סיפור משתמש, השתמשו בטכניקות בהמשך



להלן סיכום

להלן תבנית ליצירת סיפור משתמש (ללא קשר לפרומפטים)

[1. הקשר - CONTEXT]

הסביבה: _____
המצב הנוכחי: _____
האתגרים: _____

[2. תפקידים - AS A / ROLE]

אתה ה-AI _____ מומחה ב-_____.
משתמש 1 _____ שצריך _____.
משתמש 2 _____ שצריך _____.
משתמש N _____ שצריך _____.

[3. מטרה - SO THAT / GOAL]

לספק _____ (ערך) על ידי ביטול
_____ (בזבוז).

[4. משימות- I WANT]

פעולה 1: _____
פעולה 2: _____
פעולה 3: _____
פעולה N: _____

[5. קריטריוני הצלחה - ACCEPTANCE CRITERIA]

קריטריונים חיוביים _____
מה אסור _____
איכויות נדרשות _____
(למשל קצב עיבוד, זמן תגובה, חווית משתמש וכו')

[6. תרחישים -EXAMPLES]

הצלחה _____
מקרה גבולי _____
לא ולידי/ כישלון _____

רטרוספקטיבה

שלוש שאלות לסיום

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו - תפיסה
שהתהפכה, הנחה שנשברה.

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.



תודה רבה- זכרו

“

עיקרון ראשון
"ככל שקל יותר לבנות, ההחלטה האם בונים
הופכת קריטית יותר."

”

“

עיקרון שני
"בודקים את כל העלויות ולא רק את עלויות הפיתוח"

”

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

